

LasVSim Platform Installation Guide

Welcome to the "Vehicle Control Engineering" final project! This guide will walk you through the complete installation and configuration of the LasVSim simulation platforms.

1. Prerequisites

Platform: This guide is intended for **Ubuntu**.

If you do not have a stable Ubuntu environment, you can use the free computing resources provided by Tsinghua University [here](#). A CPU-only instance with 8/16 CPU cores is sufficient for this project.

Important: Please remember to back up your code frequently.

Before starting, ensure your system has **Anaconda** or **Mamba** installed. We will use `conda` or `mamba` to manage the project's Python environment. For faster environment creation, you can use `mamba`.

2. Installation Steps

Please follow these steps strictly in the order presented.

Step 1: Download the Code Repositories

Download all necessary code libraries from the Tsinghua Cloud Drive.

- **Download Link:** <https://cloud.tsinghua.edu.cn/d/cc4c748446fb45cc9c45/>

After downloading, extract the compressed file. You should see several folders, including `VCE-IDC`, `VCE-LasVSim`, and `LasVSim`.

Step 2: Install VCE-IDC (Conda Environment)

First, we will create the Conda environment and install the main `VCE-IDC` package.

```
# 1. Navigate to the VCE-IDC directory
cd path/to/VCE-IDC

# 2. Create the environment from the .yaml file
# Note: If using mamba, replace "conda" with "mamba"
conda env create -f gops_lasvsim.yaml

# 3. Activate the newly created environment
conda activate vce-idc

# 4. Install VCE-IDC in editable mode
pip install -e .
```

Step 3: Install VCE-LasVSim

While the `VCE-IDC` environment is still activated, install the `VCE-LasVSim` connector.

```
# 1. Navigate to the VCE-LasVSim directory
cd path/to/VCE-LasVSim

# 2. Install in editable mode
pip install -e .
```

Step 4: Start the `opensim` Simulation Service

`opensim` is the core `LasVSim` service that must be running as a background process.

```
# 1. Navigate to the LasVSim directory
cd path/to/LasVSim

# 2. Grant execution permissions to the script
chmod +x ./opensim

# 3. Start the service (we will use port 8290)
bash restart.sh 8290
```

Copy the port (e.g., 8290) and paste it to replace the `port` variable within the `__main__` function in the file: `./VCE-IDC/example_run/run_qx.py`.

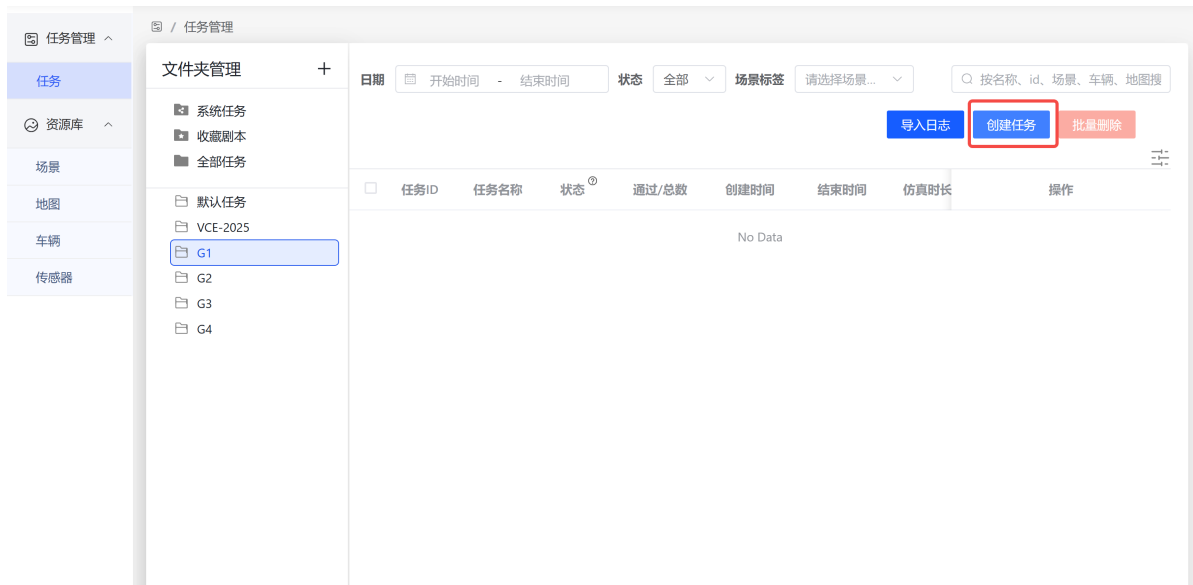
Step 5: Configure Your LasVSim Token

You need a personal Token from your LasVSim account to connect to the simulation service.

1. **Register Account:** You should have received a registration email. Contact a TA if you have not.
2. **Get Token:** Log in to the [LasVSim Platform](#) and copy your Token from the "Personal Center".
3. **Set Token:** Open the file `./VCE-LasVSim/lasvsim_env/lasvsim.token`. Paste your copied Token into this file, **completely replacing** any existing content.

Step 6: Create and Configure the Task

1. Navigate to the group folder and create a new task.

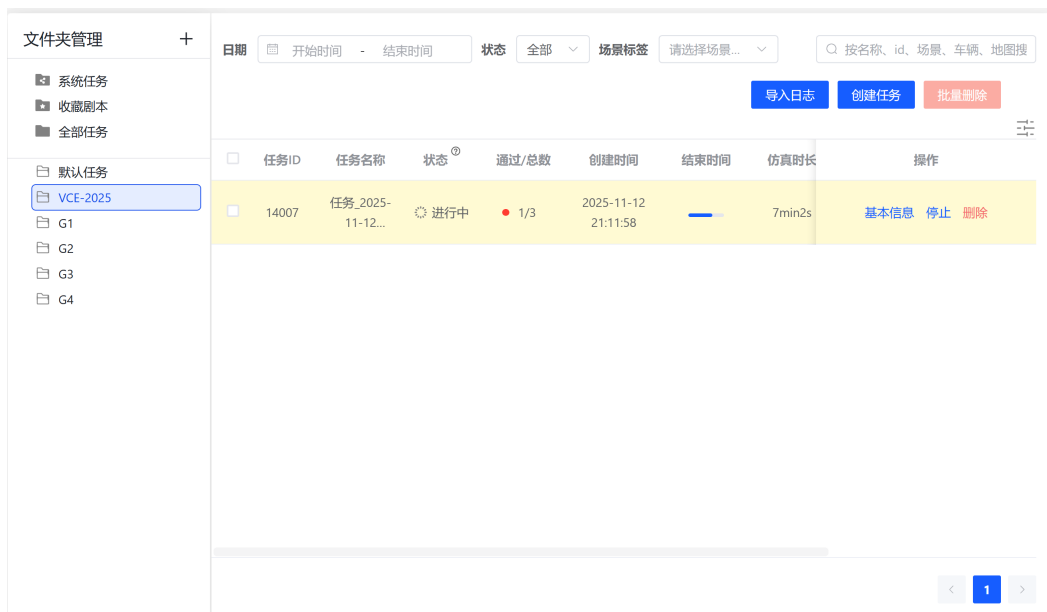


2. Fill in the task name (e.g., `{your_code_feature}_{time}`). A single task can be used for multiple tests. Set the "maximum scenario steps" to 5000, confirm the group folder, and select the latest scenario.

3. Select the latest scenario. Any changes or updates to the scenario will be communicated in the WeChat group.



- Copy the new **Task ID** (e.g., 14007) and paste it to replace the `task_id` variable within the `__main__` function in the file: `./VCE-IDC/example_run/run_qx.py`.



3. Verifying Your Installation

After completing all steps, let's verify that the environment is working correctly.

Step 6: Verify Simulation Environment Connection

This script tests the connection between your environment and the `opensim` service.

- Open the file `./VCE-LasVSim/lasvsim_env/env/env_base.py` in a text editor.
- Locate the `if __name__ == "__main__":` block at the end of the file.
- Modify the `token` and `server_host` fields with your own configuration, as shown below:

```

if __name__ == "__main__":
    from lasvsim_env.get_config import get_env_config
    env = LasvsimEnv(
        token="your_token_string_here", # 1. Paste your Token from LasVSim
        here
        env_config=get_env_config(),
        task_id=147, # 2. Your task_id
        is_testing=True,
        server_host="http://localhost:8290" # 3. Ensure the address and port
        match Step 4
    )
    env.stop_remote_LasVSim()

```

4. Execute this file **from your activated VCE-IDC environment**:

```

conda activate VCE-IDC
python ./VCE-LasVSim/lasvsim_env/env/env_base.py

```

5. If the script runs to completion **without any errors**, your simulation service is successfully connected.

Step 7: Run Testing Controller

Finally, run the example controller provided in `vce-idc` to confirm everything is working end-to-end.

```

conda activate VCE-IDC
python ./VCE-IDC/example_run/run_qx.py

```

If you see the simulation begin to run, your environment is fully installed and configured. Congratulations!

The final output will include your episode metrics and a visualization URL:

```

Episode Metrics Summary: {'safety_score': 1.0, 'compliance_score': 1.0,
'comfort_score': {'acc_diff_mean': np.float64(0.2775544449687004),
'steer_diff_mean': np.float64(0.0004545863950625062), 'acc_abs_mean':
np.float64(0.705921696126461), 'steer_abs_mean':
np.float64(0.002105202875100076)}, 'racing_score': 1.0, 'total_steps': 10}
IDC Time Stats: {'count': 10, 'mean': 0.6047636985778808, 'total':
6.047636985778809, 'min': 0.4409780502319336, 'max': 1.0273816585540771}
Average planning time: 0.6048s

```